R1.01 — Maîtrise des Structures Répétitives (Boucles), Preuve de Terminaison et Invariants

Thème principal : Algorithmique Structurée : Itération (POUR, TANT QUE, RÉPÉTER), Variant de Boucle (Terminaison) et Invariant de Boucle (Correction)

Synthèse exécutive : Ce cours pose les fondements algorithmiques de la programmation structurée à travers le concept d'itération. L'objectif est d'éliminer définitivement l'anti-pattern du copier-coller d'instructions afin de traiter des volumes de données dynamiques. Il établit une typologie stricte des boucles (bornées vs non bornées, pré-condition vs post-condition) et fournit les outils mathématiques formels d'assurance qualité logicielle : le variant de boucle (qui garantit l'absence de boucle infinie par preuve de terminaison) et l'invariant de boucle (qui garantit mathématiquement l'exactitude du résultat par preuve de correction).

1. Typologie et Choix des Structures Itératives

En algorithmique, une structure itérative permet de répéter un bloc d'instructions sans recopier textuellement ce bloc dans le code source. Le choix d'une structure itérative repose sur deux critères clairs : 1) Connaît-on le nombre d'itérations à l'avance ? Si oui, on privilégie la boucle POUR, qui prend en charge automatiquement la gestion de la variable de contrôle. 2) Si le nombre de tours dépend d'un événement dynamique (ex: saisie utilisateur, lecture de fichier, convergence de calcul), on utilise une boucle non bornée. Si le traitement peut être totalement ignoré dès le départ selon l'état initial des variables, la boucle TANT QUE s'impose (0 à N itérations). Si le traitement doit impérativement s'exécuter au moins une fois avant d'évaluer la condition de sortie (ex: affichage d'un menu interactif ou contrôle de saisie), on emploie la boucle RÉPÉTER... JUSQU'À (1 à N itérations).

- Anti-pattern du Copier-Coller : Dupliquer des lignes de code rend le programme rigide, inmaintenable et incapable d'absorber des volumes de données variables.
- Itération Bornée (POUR) : Structure privilégiée lorsque le nombre d'itérations est connu avant d'entrer dans la boucle. La variable de contrôle (compteur) est initialisée, incrémentée et testée automatiquement.
- Itération Non Bornée à Pré-condition (TANT QUE) : La condition logique est évaluée AVANT chaque passage. Si la condition est fausse dès l'entrée, le bloc s'exécute 0 fois (0 à N exécutions).
- Itération Non Bornée à Post-condition (RÉPÉTER... JUSQU'À) : Le bloc d'instructions est exécuté D'ABORD, puis la condition d'arrêt est évaluée. Garantit au moins 1 exécution (1 à N exécutions).

Cas concret enseignant : *L'analogie du coureur sur la piste d'athlétisme : Si le coach dit 'Fais 10 tours de piste', c'est une boucle POUR (nombre d'itérations fixe et déterminé d'avance). S'il dit 'Cours tant qu'il ne pleut pas', c'est une boucle TANT QUE : s'il pleut déjà lorsqu'il est au vestiaire, le coureur ne commence même pas (0 tour effectué). S'il dit 'Fais un tour et arrête-toi dès que tu es essoufflé', c'est une boucle RÉPÉTER... JUSQU'À : le coureur fait obligatoirement son premier tour avant d'évaluer son niveau d'essoufflement.*

2. Preuve Formelle de Terminaison : Le Variant de Boucle

Pour prouver scientifiquement qu'un algorithme ne restera pas bloqué indéfiniment dans une boucle non bornée, l'informaticien doit formaliser un variant de boucle. Mathématiquement, la preuve s'appuie sur une propriété fondamentale des entiers : il est impossible de construire une suite d'entiers qui soit infinie, strictement décroissante et minorée. Pour constituer une preuve de terminaison valide, deux conditions doivent être démontrées : 1) La décroissance stricte du variant V à chaque itération (V_{k+1} < V_k). 2) La minoration de V par une constante (généralement 0) combinée au fait que V <= 0 implique la sortie immédiate de la boucle. Si ces deux conditions sont vérifiées, la terminaison en un nombre fini d'étapes est formellement garantie.

- Boucle Infinie : Dysfonctionnement majeur où la condition de sortie n'est jamais satisfaite, gelant le programme ou saturant le processeur.

• Variant de Boucle : Grandeur entière dépendante des variables de l'algorithme qui exprime la 'distance' restant à parcourir avant l'arrêt.

• Décroissance Stricte : Propriété imposant au variant de diminuer d'au moins 1 entier à chaque passage dans la boucle (V_{k+1} <= V_k - 1).

• Minoration : Seuil plancher entier M (généralement 0) en dessous duquel le variant ne peut descendre sans que la condition de continuation ne devienne fausse.

• Théorème de Terminaison : Toute suite d'entiers naturels strictement décroissante et minorée est nécessairement finie (principe du bon ordre des entiers naturels).

Cas concret enseignant : *La métaphore du réservoir d'essence d'une voiture : À chaque kilomètre parcouru (itération), la quantité de carburant dans le réservoir (le variant V) diminue obligatoirement d'une quantité stricte. Comme le réservoir ne peut pas descendre en dessous de 0 litre (le minorant), la voiture finira obligatoirement par s'immobiliser. Si la jauge ne baissait pas à chaque kilomètre, la voiture roulerait à l'infini : c'est l'équivalent exact du bug de la boucle infinie.*

3. Preuve Formelle de Correction : L'Invariant de Boucle

Savoir qu'un algorithme s'arrête (terminaison) est indispensable, mais garantir qu'il calcule le bon résultat (correction) l'est tout autant. L'invariant de boucle est l'analogue de la démonstration par récurrence appliquée aux structures répétitives. On définit une propriété logique P reliant les variables du programme. La preuve se décompose en trois étapes formelles : 1) Initialisation : P est vrai avant d'entrer dans la boucle. 2) Conservation : sous l'hypothèse que P est vrai au début du tour k, on démontre que l'exécution des instructions de la boucle préserve la vérité de P pour le tour k+1. 3) Conclusion : lorsque la boucle se termine, la condition de maintien C devient fausse (not C). La conjonction logique (P et not C) permet de déduire mathématiquement que la variable de sortie contient la valeur exacte exigée par la spécification.

• Invariant de Boucle : Predicat ou assertion logique P portant sur les variables du programme qui demeure VRAI avant le début de la boucle, après chaque itération et à la sortie.

• Phase d'Initialisation : Démonstration que l'invariant P est vrai dès l'état initial des variables, juste avant d'effectuer le premier tour.

• Phase de Conservation (Hérédité) : Preuve que si P est vrai au début d'une itération k arbitraire, il demeure rigoureusement vrai à la fin du tour k après la mise à jour des variables.

• Phase de Conclusion (Correction) : Combinaison logique de l'invariant conservé (P) et de la négation de la condition de boucle (non C) prouvant que la valeur finale calculée est exacte.

Cas concret enseignant : *L'analogie de la tournée du livreur de journaux : Au départ de sa tournée matinale, l'égalité 'Journaux distribués + Journaux restants dans le sac = Total initial' est vraie. À chaque maison servie, les deux quantités changent mais leur somme reste rigoureusement la même tout au long de la journée (invariant). Quand le sac est vide à la fin de la tournée (condition de sortie), le nombre de journaux distribués est obligatoirement égal au total initial.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Est-ce qu'on peut toujours remplacer une boucle POUR par une boucle TANT QUE ?
Réponse du professeur : Oui, absolument. Toute boucle POUR peut être traduite fidèlement par une boucle TANT QUE : il suffit d'initialiser le compteur avant la boucle, d'exprimer la condition de maintien (compteur <= borne_fin) et d'incrémenter manuellement le compteur à la fin du corps de la boucle. En revanche, la boucle POUR est à privilégier lorsque le nombre d'itérations est fixe, car elle gère le compteur automatiquement et évite les oublis d'incrémentation qui causent des boucles infinies.

Question étudiant : Si j'utilise une boucle TANT QUE avec la condition (i < N), est-ce que le corps s'exécute si i est déjà égal à N dès le départ ?
Réponse du professeur : Non. La boucle TANT QUE est à pré-condition : la condition de maintien (i < N) est évaluée AVANT toute exécution du corps. Si i est égal à N dès le départ, la condition (N < N) évalue à Faux immédiatement. Le corps de la boucle est donc totalement ignoré et s'exécute exactement 0 fois.

■ Exercice & Démonstration d'Application

Énoncé : Écrire un algorithme en pseudo-code permettant de calculer la somme des N premiers entiers pairs stricts (par exemple pour N=3, calculer 2 + 4 + 6 = 12). Justifier le choix de la boucle, rédiger le code propre et formaliser le variant de boucle garantissant la terminaison.

Résolution pas à pas :
Étape 1 : Choix de la structure itérative. Le paramètre N est fourni en entrée du programme ; le nombre total d'itérations est donc parfaitement déterminé et connu d'avance. La boucle POUR est la structure la plus adaptée et la plus sûre. Étape 2 : Définition et initialisation des variables. - N : Entier (nombre de termes à sommer) - i : Entier (compteur d'itérations de 1 à N) - somme : Entier (accumulateur, initialisé à 0) - terme : Entier (valeur du terme pair courant, initialisé à 2) Étape 3 : Rédaction du pseudo-code formel. Algorithme SommePremiersPairs Variables : N, i, somme, terme : Entiers Début Lire(N) somme <- 0 terme <- 2 Pour i allant de 1 à N par pas de 1 Faire somme <- somme + terme terme <- terme + 2 FinPour Afficher(somme) Fin Étape 4 : Démonstration du variant de boucle. Si l'on traduit cette itération sous forme de boucle TANT QUE équivalente contrôlée par le compteur i (initialisé à 1 avec condition i <= N), la quantité V = (N - i + 1) constitue un variant de boucle valide : 1. À chaque itération k, i augmente de 1, donc V_{k+1} = N - (i + 1) + 1 = V_k - 1 (décroissance stricte d'au moins 1). 2. La boucle continue tant que i <= N (soit V >= 1). Dès que V atteint 0 (minorant), la condition devient fausse et la boucle s'arrête.

Résultat : Pour N = 3 : - Tour 1 (i=1) : somme = 0 + 2 = 2 ; terme devient 4. - Tour 2 (i=2) : somme = 2 + 4 = 6 ; terme devient 6. - Tour 3 (i=3) : somme = 6 + 6 = 12 ; terme devient 8. Sortie de boucle. Valeur affichée = 12. L'algorithme se termine de manière scientifiquement prouvée en exactement N itérations et produit le résultat exact.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la différence fondamentale d'exécution entre une boucle TANT QUE et une boucle RÉPÉTER... JUSQU'À ?

R : La boucle TANT QUE évalue sa condition AVANT d'exécuter le corps (pré-condition, de 0 à N exécutions possible). La boucle RÉPÉTER... JUSQU'À évalue sa condition APRÈS l'exécution du corps (post-condition), ce qui garantit que le bloc d'instructions s'exécutera au moins 1 fois (1 à N exécutions).

Q : Quelles sont les deux propriétés obligatoires qu'une quantité entière doit vérifier pour constituer un variant de boucle valide ?

R : 1) Elle doit être strictement décroissante à chaque itération (V_{k+1} < V_k). 2) Elle doit être minorée par une constante entière (généralement 0) en dessous de laquelle la condition de boucle devient fausse.

Q : Quel oubli classique dans la rédaction d'une boucle TANT QUE provoque une boucle infinie et fait perdre des points en DS de R1.01 ?

R : L'oubli de la mise à jour ou de l'incrémentation de la variable de contrôle dans le corps de la boucle (ex: oublier i <- i + 1). La condition de maintien reste alors perpétuellement vraie, bloquant le programme à l'infini.